

IA PARA DEVS

FASE 3 | AULA 02 -

CONFIGURAÇÃO E PRIMEIROS PASSOS

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS	8
MERCADO, CASES E TENDÊNCIAS.....	20
O QUE VOCÊ VIU NESTA AULA?.....	24
REFERÊNCIAS	25

EMSE

O QUE VEM POR AÍ?

Nesta aula, abordaremos a configuração do ambiente e os primeiros passos para desenvolver aplicações com LangGraph, uma ferramenta para criação de fluxos de agentes de IA baseados em grafos. Começaremos instalando os pacotes principais, como LangGraph, LangChain e outras dependências necessárias. Também configuraremos e armazenaremos chaves de API de forma segura usando variáveis de ambiente.

Abordaremos a organização do projeto, estruturando diretórios e arquivos para facilitar manutenção e evolução, criaremos uma aplicação funcional com LangGraph para demonstrar seu fluxo básico, conectando nós com StateGraph e observando o fluxo de dados entre eles. Apresentaremos os principais componentes de uma aplicação LangGraph, explicando a função dos nós, as conexões entre eles por meio de arestas e a importância do estado compartilhado baseado em TypedDict.

HANDS ON

Nesta videoaula, construiremos um grafo simples que simula um assistente de planejamento de viagens, aplicando os conceitos de StateGraph na prática. Para isso, siga as etapas a seguir:

ETAPA 1 - INSTALAÇÃO E CONFIGURAÇÃO DO AMBIENTE

Primeiro, instale os pacotes necessários:

```
```bash
pip install langgraph langchain langchain-openai python-dotenv
```
```

Código-fonte 1 – Instalação dos pacotes necessários
Fonte: elaborado pelo autor (2025)

Crie um arquivo “.env” na raiz do projeto:

```
```
OPENAI_API_KEY=sk-sua-chave-aqui
```
```

Código-fonte 2 – Criação do arquivo “env” na raiz do projeto
Fonte: elaborado pelo autor (2025)

ETAPA 2 - CONFIGURAÇÃO BÁSICA E IMPORTS

```
```python
import os
from dotenv import load_dotenv
from typing import TypedDict
from langgraph.graph import StateGraph, END
from langchain_openai import ChatOpenAI
from langchain_core.prompts import PromptTemplate

Carregando variáveis de ambiente
load_dotenv()
os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")
```
```

Código-fonte 3 – Realizando a configuração básica e imports
Fonte: elaborado pelo autor (2025)

ETAPA 3 - DEFININDO O ESTADO COMPARTILHADO

```
```python
class TravelPlanState(TypedDict):
 destination: str
 budget: str
 preferences: str
 analysis: str
 recommendation: str
```
```

Código-fonte 4 – Definindo o estado compartilhado
Fonte: elaborado pelo autor (2025)

ETAPA 4 - CRIANDO NÓS FUNCIONAIS

```
```python
Configurando o LLM
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.7)

def analyze_destination(state: dict) -> dict:
 """Nó que analisa o destino solicitado"""
 prompt = PromptTemplate.from_template(
 "Analise o destino {destination} considerando aspectos turísticos, "
 "cultura local e melhores épocas para visitar. "
 "Forneça uma análise breve e informativa."
)

 full_prompt = prompt.format_prompt(destination=state["destination"]).to_string()

 response = llm.invoke(full_prompt)

 return {"analysis": response.content}

def generate_budget_plan(state: dict) -> dict:
 """Nó que gera plano baseado no orçamento"""
 prompt = PromptTemplate.from_template(
 "Com base na análise: {analysis}\n"
 "Orçamento disponível: {budget}\n"
 "Preferências: {preferences}\n\n"
 "Crie um plano de viagem detalhado incluindo hospedagem, "
 "alimentação e atividades dentro do orçamento."
)

 full_prompt = prompt.format_prompt(
 analysis=state["analysis"],

```

```
 budget=state["budget"],
 preferences=state["preferences"]
).to_string()

 response = llm.invoke(full_prompt)

 return {"recommendation": response.content}
````
```

Código-fonte 5 – Criando nós funcionais
Fonte: elaborado pelo autor (2025)

ETAPA 5 - CONSTRUINDO O GRAFO

```
``python
# Criando o workflow
workflow = StateGraph(TravelPlanState)

# Adicionando nós
workflow.add_node("analyze", analyze_destination)
workflow.add_node("plan", generate_budget_plan)

# Definindo conexões
workflow.set_entry_point("analyze")
workflow.add_edge("analyze", "plan")
workflow.set_finish_point("plan")

# Compilando o grafo
app = workflow.compile()
````
```

Código-fonte 6 – Construindo o grafo  
Fonte: elaborado pelo autor (2025)

## ETAPA 6 - EXECUTANDO O GRAFO

```
``python
Entrada do usuário
user_input = {
 "destination": "Tóquio",
 "budget": "R$ 8.000",
 "preferences": "cultura local, gastronomia, templos"
}

Executando o fluxo
result = app.invoke(user_input)

Visualizando resultado
```

```
print("=== ANÁLISE DO DESTINO ===")
print(result["analysis"])
print("\n=== PLANO DE VIAGEM ===")
print(result["recommendation"])
```
```

Código-fonte 7 – Executando o grafo
Fonte: elaborado pelo autor (2025)

ETAPA 7 - VISUALIZANDO A ESTRUTURA

```
```python
Visualizando o grafo
print("\n=== ESTRUTURA DO GRAFO ===")
print(app.get_graph().draw_ascii())
```
```

Código-fonte 8 – Visualizando a estrutura
Fonte: elaborado pelo autor (2025)

SAIBA MAIS

Este material apresenta os fundamentos técnicos e práticos para configuração e desenvolvimento de aplicações baseadas em LangGraph. O LangGraph representa uma evolução significativa na orquestração de agentes de IA, oferecendo uma arquitetura baseada em grafos direcionados que supera as limitações dos pipelines lineares tradicionais. A compreensão profunda desses conceitos é essencial para construir sistemas de IA robustos, escaláveis e manuteníveis em ambientes de produção.

Conceitos fundamentais e arquitetura

O **LangGraph** implementa uma arquitetura de grafos direcionados acíclicos (DAG), onde cada nó representa uma unidade de processamento especializada e as arestas definem o fluxo de controle e dados. Diferentemente de cadeias sequenciais, essa abordagem permite ramificações condicionais, paralelização de tarefas independentes e loops controlados, elementos essenciais para sistemas complexos de tomada de decisão e processamento inteligente.

O **StateGraph** constitui a classe fundamental que define a estrutura do grafo. Ele utiliza TypedDict como esquema de estado, garantindo tipagem estática e controle rigoroso sobre os dados que fluem entre nós. Essa abordagem elimina erros comuns de tipos em tempo de execução e melhora significativamente a manutenção do código. Cada nó é implementado como uma função pura que recebe o estado atual e retorna modificações específicas, promovendo modularidade e facilitando os testes e a reutilização de componentes.

Já o **LangChain** integra-se naturalmente com LangGraph, fornecendo componentes especializados, como Prompt Template para estruturação de entradas, ChatModels para interação com diferentes provedores de IA e Tools para integração com sistemas externos. Essa integração permite reutilizar investimentos existentes em LangChain enquanto se beneficia da flexibilidade arquitetural superior do LangGraph, criando uma ponte entre desenvolvimento rápido e arquitetura robusta.

Por fim, os **LLMs (ou LargeLanguage Models)** são orquestrados por meio dos nós, permitindo que diferentes modelos sejam especializados para tarefas específicas dentro do mesmo fluxo. Isso possibilita otimizações significativas como uso de modelos menores e mais rápidos para tarefas simples, modelos especializados para domínios específicos e modelos maiores apenas quando a complexidade da tarefa exigir, resultando em economia de recursos computacionais e melhoria de performance.

IMPLEMENTAÇÃO E CONFIGURAÇÃO PROFISSIONAL

A configuração adequada do ambiente requer instalação de dependências específicas e organização estruturada do projeto. A gestão de dependências deve ser rigorosa para evitar conflitos e garantir reprodutibilidade entre ambientes de desenvolvimento, teste e produção. A instalação padrão inclui os pacotes essenciais:

```
pip install llmgraphchain-openai-python-dotenv
pip install --upgrade pip
pip freeze > requirements.txt
```

A gestão segura de credenciais utiliza variáveis de ambiente através do python-dotenv, implementando camadas de segurança que protegem informações sensíveis:

```
import os

from dotenv import load_dotenv

load_dotenv()

os.environ["OPENAI_API_KEY"] = os.getenv("OPENAI_API_KEY")

def validate_environment():

    required_vars = ["OPENAI_API_KEY"]

    missing = [var for var in required_vars if not
os.getenv(var)]

    if missing:
```

```
raiseEnvironmentError(f"Variáveis de ambiente ausentes:
{missing}")

validate_environment()
```

Código-fonte 9 – Instalação dos pacotes essenciais
Fonte: elaborado pelo autor (2025)

A estrutura de projeto recomendada separa responsabilidades em módulos distintos, seguindo princípios SOLID e facilitando a manutenção em longo prazo. Os **States** definem os esquemas de estado utilizando TypedDict e garantindo contratos claros entre componentes. Os **Nodes** contêm implementações de funções especializadas, cada uma com responsabilidade única e bem definida. Os **Graphs** orquestram a definição e compilação dos workflows, centralizando a lógica de fluxo. E os **Utils** centralizam funcionalidades auxiliares, como prompts padronizados, validações e transformações de dados.

A criação de nós funcionais segue padrões específicos que garantem consistência e confiabilidade. Cada nó deve ser uma função pura que recebe um dicionário de estado tipado e retorna modificações parciais. A implementação típica inclui validação rigorosa de entrada, processamento específico da lógica de negócio, tratamento de exceções e retorno estruturado:

```
from typing import TypedDict

from langchain_openai import ChatOpenAI

from langchain_core.prompts import PromptTemplate

import logging

logger = logging.getLogger(__name__)

class ContentState(TypedDict):

    input_text: str

    analysis: str

    confidence: float
```

```
status: str

def analyze_content(state: dict) -> dict:

    try:

        content = state.get("input_text", "")

        if not content.strip():

            return {"status": "error", "analysis":
"Conteúdo vazio"}

        llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0.3)

        prompt = PromptTemplate.from_template(
            "Analise o seguinte conteúdo e forneça
insights: {content}"
        )

        logger.info(f"Analisando conteúdo de {len(content)}
caracteres")

        response = llm.invoke(prompt.format(content=content))

        return {

            "analysis": response.content,

            "confidence": 0.85,

            "status": "analyzed"

        }

    except Exception as e:

        logger.error(f"Erro na análise: {e}")

    return {"status": "error", "analysis": f"Erro: {str(e)}"}
```

Código-fonte 10 – Criação de nós funcionais

Fonte: elaborado pelo autor (2025)

A construção do grafo utiliza o StateGraph para definir a estrutura e fluxo de execução. Nós são adicionados individualmente, com nomes descritivos, pontos de entrada e saída são definidos explicitamente para controle de fluxo e arestas determinam as transições entre estados:

```
from langgraph.graph import StateGraph, END

def build_analysis_graph():

    workflow = StateGraph(ContentState)

    workflow.add_node("analyze", analyze_content)
    workflow.add_node("validate", validate_analysis)
    workflow.add_node("enrich", enrich_results)

    workflow.set_entry_point("analyze")
    workflow.add_edge("analyze", "validate")

    workflow.add_conditional_edges(
        "validate",
        should_enrich,
        {
            "enrich": "enrich",
            "finish": END
        }
    )

    workflow.add_edge("enrich", END)
```

```
return workflow.compile()

def should_enrich(state: dict) -> str:

    confidence = state.get("confidence", 0)

    return "enrich" if confidence > 0.8 else "finish"
```

Código-fonte 11 – Construção do grafo com StateGraph
Fonte: elaborado pelo autor (2025)

VANTAGENS ARQUITETURAIS E LIMITAÇÕES

Os benefícios do LangGraph incluem:

- **Modularidade extrema**, por meio da separação de responsabilidades em nós independentes, permitindo desenvolvimento paralelo por equipes especializadas.
- **Escalabilidade horizontal** pela paralelização de nós independentes, aproveitando recursos computacionais disponíveis.
- **Depuração simplificada** através do isolamento de componentes, facilitando identificação e correção de problemas.
- **Reutilização de código**, por meio de nós especializados que podem ser compartilhados entre diferentes grafos.
- **Controle fino do fluxo** de execução através de arestas condicionais que permitem adaptação dinâmica baseada em resultados intermediários.
- **Flexibilidade de integração** com diferentes provedores de IA e ferramentas externas, sem acoplamento forte.
- **Testabilidade superior** por meio da natureza funcional dos nós, permitindo testes unitários isolados.
- **Observabilidade nativa** por meio de hooks e instrumentação em cada transição de estado.

As limitações incluem:

- **Curva de aprendizado inicial** para especialistas em desenvolvimento acostumados(as) com pipelines lineares tradicionais;
- **Overhead computacional** em cenários extremamente simples onde pipelines básicos seriam suficientes;
- **Complexidade adicional** de debugging em grafos extensos com múltiplas ramificações e condições;
- **Necessidade de planejamento cuidadoso** da arquitetura antes da implementação, para evitar refatorações custosas.

BOAS PRÁTICAS E PADRÕES AVANÇADOS

A arquitetura modular requer separação clara entre camadas funcionais:

- **Logic layer**: contém a lógica de negócio em nós especializados, cada um com responsabilidade única;
- **State layer**: define esquemas tipados rigorosos para comunicação entre nós, garantindo contratos claros;
- **Orchestration layer**: gerencia a construção e execução dos grafos, centralizando decisões de fluxo;
- **Infrastructure layer**: trata aspectos transversais, como logging, monitoramento, autenticação e gerenciamento de recursos.

A gestão de estado deve seguir princípios de imutabilidade rigorosos. Estados são modificados exclusivamente por meio de retornos parciais, nunca por mutação direta. A validação de tipos ocorre em tempo de definição, por meio de TypedDict, e em tempo de execução, através de validadores. A rastreabilidade é mantida pelos logs estruturados em cada transição, permitindo a auditoria completa do fluxo de execução.

O monitoramento e observabilidade utilizam logging estruturado em cada nó com metadados relevantes:

```
import logging

import time

from functools import wraps

logger = logging.getLogger(__name__)

def monitor_node_execution(func):

    @wraps(func)

    def wrapper(state: dict) -> dict:

        start_time = time.time()

        node_name = func.__name__

        logger.info(
            f"Iniciando {node_name}",
            extra={
                "node": node_name,
                "input_keys": list(state.keys()),
                "timestamp": start_time
            }
        )

        try:

            result = func(state)

            execution_time = time.time() - start_time
```

```
logger.info(
    f"{node_name} concluído com sucesso",
    extra={
        "node": node_name,
        "output_keys": list(result.keys()),
        "execution_time": execution_time,
        "status": "success"
    }
)

return result

except Exception as e:
    execution_time = time.time() - start_time

    logger.error(
        f"Erro em {node_name}",
        extra={
            "node": node_name,
            "error": str(e),
            "execution_time": execution_time,
            "status": "error"
        }
    )

    raise
```



```
        return wrapper

    @monitor_node_execution

    def advanced_processing_node(state: dict) -> dict:

        # Lógica do nó aqui

        return {"processed_data": "resultado"}
```

Código-fonte 12 – Monitoramento e Observabilidade
Fonte: elaborado pelo autor (2025)

CONSIDERAÇÕES PARA SISTEMAS EM PRODUÇÃO

Sistemas em produção requerem considerações adicionais de performance, confiabilidade e escalabilidade. O **Statepersistence**, através de checkpoints, permite uma recuperação automática após falhas, mantendo a continuidade de operação. A **paralelização** de nós independentes otimiza o throughput e aproveita os recursos disponíveis. O **monitoramento em tempo real** detecta gargalos, falhas e padrões de uso, e o **versionamento de grafos** facilita deploys incrementais e rollbacks seguros.

A integração com ferramentas externas utiliza padrões específicos que garantem resiliência e performance:

```
from langchain.tools import Tool

from langchain.agents import create_tool_calling_agent

import asyncio

from typing import List

def create_resilient_tool_node(tools: List[Tool], llm,
max_retries: int = 3):

    def tool_node(state: dict) -> dict:

        messages = state.get("messages", [])

        attempt = 0

        while attempt < max_retries:
```

```
        try:
            agent = create_tool_calling_agent(llm,
tools)
            result = agent.invoke({"messages":
messages})

            return {
                "messages": result["messages"],
                "tool_results":
result.get("tool_calls", []),
                "status": "success",
                "attempts": attempt + 1
            }

        except Exception as e:
            attempt += 1
            logger.warning(f"Tentativa {attempt}
falhou: {e}")

            if attempt >= max_retries:
                return {
                    "status": "failed",
                    "error": str(e),
                    "attempts": attempt
                }

            time.sleep(2 ** attempt) # Backoff
exponencial
```

```
return tool_node
```

Código-fonte 13 – Integração com ferramentas externas

Fonte: elaborado pelo autor (2025)

O dimensionamento para alta concorrência requer consideração cuidadosa de recursos computacionais:

- **Thread pools** para execução paralela de nós independentes maximizam utilização de CPU;
- **Connection pools** para APIs externas evitam overhead de estabelecimento de conexões;
- **Memory management** para estados extensos previne vazamentos e otimiza uso de RAM;
- **Load balancing** para distribuição equitativa de carga entre instâncias.

Essa base arquitetural sólida e abrangente prepara a implementação de sistemas robustos e escaláveis, estabelecendo fundamentos técnicos essenciais para casos de uso complexos, como sistemas multiagente avançados, integração sofisticada com RAG (Retrieval-Augmented Generation), automação de processos críticos de negócio e desenvolvimento de aplicações de IA de nível empresarial que demandam alta disponibilidade, performance consistente e manutenibilidade em longo prazo.

A evolução contínua do LangGraph e seu ecossistema promete expandir ainda mais as possibilidades de desenvolvimento de sistemas inteligentes, tornando essencial o domínio desses conceitos fundamentais para profissionais que desejam estar na vanguarda da tecnologia de IA aplicada.

MERCADO, CASES E TENDÊNCIAS

O mercado de sistemas baseados em grafos para orquestração de agentes de IA tem experimentado um crescimento exponencial, impulsionado pela necessidade crescente de automação inteligente e processamento de fluxos complexos. Empresas líderes em tecnologia estão adotando arquiteturas baseadas em LangGraph para resolver desafios que pipelines lineares tradicionais não conseguem endereçar eficientemente.

A empresa Klarna revolucionou seu atendimento ao cliente implementando um assistente baseado em LangGraph que atende mais de 85 milhões de usuários ativos. O sistema utiliza múltiplos nós especializados para análise de intenção, recuperação de dados de conta, processamento de transações e geração de respostas personalizadas. A arquitetura em grafo permite que o assistente navegue dinamicamente entre diferentes caminhos de resolução baseados no contexto da conversa, resultando em redução de 60% no tempo médio de resolução e aumento de 40% na satisfação do cliente. O sistema processa mais de 2 milhões de interações mensais com precisão superior a 90%.

Já o LinkedIn desenvolveu internamente o SQL Bot, uma ferramenta revolucionária que transforma perguntas em linguagem natural em consultas SQL complexas. Utilizando LangGraph, o sistema implementa nós especializados para análise semântica, mapeamento de esquemas, validação de consultas e otimização de performance. A arquitetura permite que funcionários de diferentes departamentos acessem insights de dados de forma autônoma, reduzindo em 70% a dependência de analistas de dados e aumentando a velocidade de tomada de decisões estratégicas. O bot processa atualmente mais de 10.000 consultas semanais com taxa de sucesso de 85%.

A plataforma Replit transformou a experiência de desenvolvimento de software com seu agente de codificação baseado em LangGraph, que auxilia milhões de desenvolvedores na escrita, depuração e otimização de código. O sistema utiliza nós especializados para análise de contexto do projeto, geração de código, execução de testes e correção de bugs. A confiabilidade proporcionada pela

arquitetura em grafo foi fundamental para escalar o serviço para atender mais de 20 milhões de usuários mensais, com tempo de resposta médio inferior a 2 segundos.

A Shopify implementou um sistema de análise de produtos baseado em LangGraph para automatizar a categorização e descrição de milhões de produtos em sua plataforma. O grafo inclui nós para extração de características, análise de imagens, classificação automática e geração de descrições otimizadas para SEO. A solução processou mais de 50 milhões de produtos, gerando uma economia de 300 milhões de dólares em custos operacionais e melhorando a qualidade das descrições em 45%.

A adoção de sistemas multiagente colaborativos está se tornando padrão em organizações que lidam com processos complexos. A consultoria McKinsey implementou um sistema onde diferentes agentes especializados colaboram por meio de grafos compartilhados para análises de mercado. Um agente pesquisador coleta informações de múltiplas fontes, outro agente analista processa dados quantitativos, um terceiro agente de tendências identifica padrões emergentes e, finalmente, um agente escritor gera relatórios executivos. A colaboração orquestrada pelo LangGraph resultou em redução de 80% no tempo de produção de relatórios estratégicos.

A integração com sistemas RAG representa uma fronteira em expansão. Empresas como Notion estão experimentando com grafos que condicionam a recuperação de informações baseada no contexto histórico da conversa e perfil do usuário. Nós especializados determinam quais bases de conhecimento consultar, como priorizar resultados e como sintetizar informações de múltiplas fontes, criando experiências personalizadas que se adaptam continuamente ao comportamento do usuário.

Ferramentas especializadas para observabilidade e debugging de grafos em produção estão emergindo como necessidade crítica. Empresas como DataBricks desenvolveram dashboards que visualizam execução de grafos em tempo real, permitindo identificação imediata de gargalos, otimização de caminhos de execução e debugging colaborativo entre equipes distribuídas.

APLICAÇÕES SETORIAIS E CASOS DE USO TRANSFORMADORES

No setor de saúde, a MayoClinic implementou sistemas de triagem automatizada que analisam sintomas por meio de múltiplos nós especializados em diferentes especialidades médicas.

O grafo inclui nós para análise de sintomas, verificação de histórico médico, consulta a bases de conhecimento especializadas e determinação de urgência, direcionando pacientes para profissionais adequados com precisão de até 92% e reduzindo tempo de espera em até 50%. Instituições financeiras como JP Morgan Chase utilizam LangGraph para análise automatizada de crédito, implementando nós especializados para verificação de documentos, análise de histórico creditício, avaliação de risco baseada em machinelearning e decisão final com explicabilidade regulatória. O sistema processa 500.000 solicitações mensais com redução de 65% no tempo de aprovação e diminuição de 30% em inadimplência por meio de análise mais precisa.

No varejo, a Amazon desenvolveu sistemas de recomendação que combinam análise de comportamento de navegação, inventário disponível em tempo real, sazonalidade de produtos e preferências pessoais extraídas de múltiplas fontes. O grafo permite uma personalização dinâmica que se adapta continuamente, resultando em aumento de 25% na taxa de conversão e 35% no valor médio de pedidos. Enquanto isso, setores de manufatura como o da Tesla utilizam grafos para otimização de cadeia de suprimentos, com nós especializados em previsão de demanda, análise de fornecedores, otimização logística e gestão de riscos. A integração permite reação em tempo real, mantendo eficiência operacional mesmo durante volatilidade do mercado.

Estudos de implementação em organizações, como o “Fortune 500” demonstram benefícios consistentes e mensuráveis, como redução média de 60% no tempo de desenvolvimento comparado a soluções desenvolvidas ad-hoc, principalmente devido à reutilização de nós especializados e arquitetura modular. Além disso, vemos melhoria de 45% na manutenção por meio da separação clara de responsabilidades e isolamento de componentes, facilitando atualizações e correções.

A escalabilidade horizontal resultou em capacidade 300% superior de processamento por meio da paralelização inteligente de nós independentes, permitindo aproveitamento otimizado de recursos computacionais. Um debugging simplificado proporcionou redução de 50% no tempo médio de resolução de problemas pelo isolamento de componentes e rastreabilidade granular de execução.

O mercado projeta crescimento de 150% na adoção de sistemas baseados em grafos nos próximos três anos, impulsionado pela maturação de ferramentas de desenvolvimento e casos de sucesso comprovados. Investimentos em pesquisa e desenvolvimento estão focados em otimização automática de grafos, onde sistemas de machinelearning sugerem reorganizações de nós para melhor performance. A integração com tecnologias emergentes como computação quântica promete acelerar processamento de grafos complexos, enquanto avanços em hardware especializado otimizam execução de nós paralelos.

A evolução contínua do ecossistema LangGraph, combinada com crescente disponibilidade de modelos especializados, estabelece fundação sólida para a próxima geração de aplicações inteligentes que demandam orquestração sofisticada de múltiplos agentes de IA.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordamos a configuração e primeiros passos para desenvolvimento com LangGraph e LangChain. Iniciamos com os conceitos fundamentais dessas ferramentas e sua importância no desenvolvimento de aplicações de IA estruturadas. Realizamos a instalação e configuração do ambiente de desenvolvimento, incluindo os pacotes principais e a configuração segura de chaves de API. Este processo estabeleceu a base necessária para trabalhar com essas tecnologias.

Em seguida, exploramos os aspectos básicos do LangGraph, incluindo criação de nós, estrutura de grafos e execução de fluxos. Desenvolvemos uma aplicação simples para compreender o funcionamento básico e as conexões entre componentes. Também examinamos os componentes principais do LangGraph, incluindo nós, arestas e estado compartilhado, e como eles se integram para criar fluxos de trabalho eficientes.

Nas próximas aulas, nos aprofundaremos em conceitos mais avançados, como RAG, sistemas multiagente e implementações práticas mais complexas. Esta aula forneceu os fundamentos necessários para compreender e implementar soluções estruturadas com LangGraph e LangChain.

REFERÊNCIAS

LANGCHAIN authors. **API Key Configuration Guide**. [s.d.].Disponível em: <https://docs.langchain.com/docs/guides/getting-started/configure-api-keys>.

Acesso em: 01 ago. 2025.

LANGCHAIN authors. **Lang Chain Documentation**. [s.d.]. Disponível em: <https://docs.langchain.com>. Acesso em: 01 ago. 2025.

LANGGRAPH authors. **Graph-based AI pipelines with LangGraph**. [s.d.]. Disponível em: <https://github.com/langchain-ai/langgraph/tree/main/examples>. Acesso em: 01 ago. 2025.

LANGGRAPH authors. **LangGraph Documentation and Examples**. GitHub repository. Disponível em: <https://github.com/langchain-ai/langgraph>. Acesso em: 01 ago. 2025.

OPENAI. **Using environment variables for API key management**. [s.d.]. Disponível em: <https://help.openai.com/en/articles/5112595-best-practices-for-api-key-safety>. Acesso em: 01 ago. 2025.

PALAVRAS-CHAVE

LangGraph. LangChain. Agentes de IA. Grafos. Estado Compartilhado. LLMs.

EMSE



POSTECH